Lab 7

A Compact RISC-V Single-Cycle Processor

1. Introduction

In this lab you will implement a compact version of the RISC-V processor we studied in class. You will implement its single-cycle micro-architecture utilizing modules from projects that you have already implemented in previous lab assignments. You will test this processor by executing programs of RISC-V instructions . Your processor will execute the machine code generated by assembling these programs using RARS.

1.2 Simplified RISC-V Architecture

Our compact RISC-V processor can only execute four R-Type RISC-V instructions to perform the add, subtract, logic AND, and logic OR operations. The machine code format of these instructions is shown in Figure 1.

R-Type

31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	op
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Figure 1: RISC-V R-Type Format

The codes of the above instructions are listed in Table 1. All of them have the same Opcode as they all are R-Type instructions. You can distinguish them by examining the 12th, 13th, and 30th bits of their instruction code. Both bits number 12 and 13 are part of the **funct3** field, while bit number 30 is part of the **funct7** field. Therefore, the control unit of this compact processor only needs to examine these three bits of each instruction to decide on which operation to perform.

Table 1: Instructions Description and Codes

Instruction	Description	Opcode	funct3 bits: 14-12	funct7 bits: 31-25
`add rd,rs1,rs2`	`rd = rs1 + rs2`	0110011	000	000 0000
`sub rd,rs1,rs2`	`rd = rs1 - rs2`		000	010 0000
`and rd,rs1,rs2`	`rd = rs1 & rs2`		111	000 0000
`or rd,rs1,rs2`	`rd = rs1 | rs2`		110	000 0000

In lab 4 you designed an ALU that can perform all these four operations. In that design we controlled the ALU operation using 2-bit **ALUControl** input as shown in Table 2.

Table 2: ALUControl Operations

ALUControl$_1$	**ALUControl$_0$**	**Operation**
0	0	ADD
0	1	SUB
1	0	AND
1	1	OR

Pre-Lab Assignment
Based on the data presented in Table 1 and 2 above, implement two combinational circuits to generate as outputs **ALUControl$_1$** and **ALUControl$_0$**. The inputs to these circuits are bits number 12, 13, and 30 of the instruction code. Provide the truth table and the two Karnaugh maps needed to implement these circuits.

Have the prelab assignment answers ready to be checked at the beginning of the lab session.

2. Compact RISC-V Single-Cycle Microarchitecture

Figure 2 shows the block diagram of our compact RISC-V Single-Cycle microarchitecture. The following are differences between this microarchitecture and the one we studied in class.

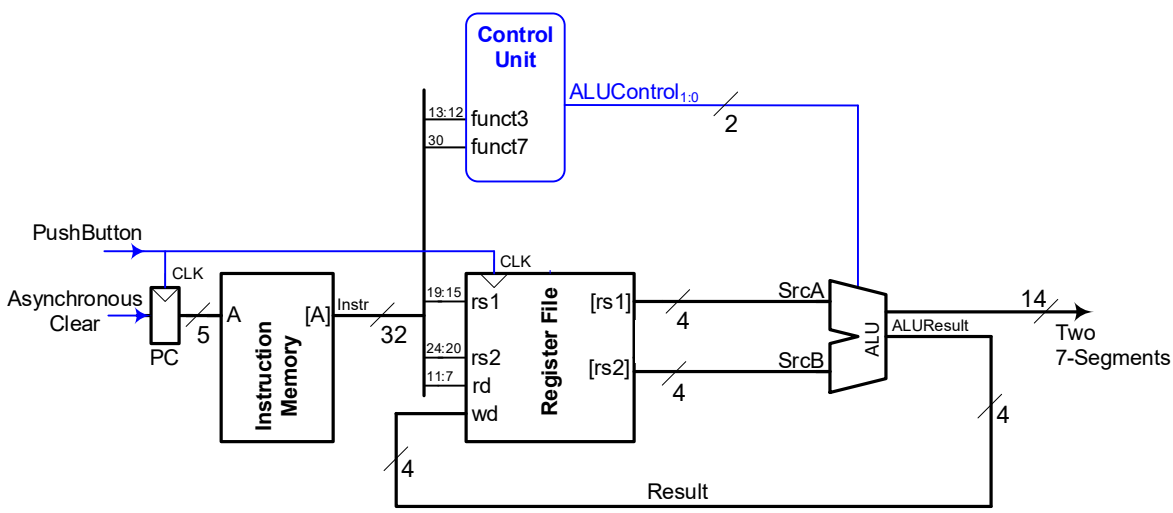

Figure 2: Microarchitecture of a Compact RISC-V Single-Cycle Processor

1) There is no data memory, as we will not handle load or store instructions.
2) A push button is used to emulate the system clock, which is used to increment the program counter (PC) and to trigger the writing process in the Register File.
3) The Instruction Memory is word addressable. This means accessing the next instruction requires adding 1 to the address of the current instruction (not 4). Therefore, there is no need for the PCPlus4 adder. Furthermore, an asynchronous clear input is used to reset the counter to zero and restart executing the program stored in the instruction memory.

4) The control unit is simplified to be just the combinational circuits from the pre-lab assignment.
5) There is no an Extend unit as we do not handle any immediate fields.
6) All modules and signals that deal with the branch or jump instructions are removed.
7) Writing to the register file is always enabled (as all our four instructions write to a register). Therefore, there is no need for a write enable (WE) input signal to the register file.
8) The registers are only 4-bit wide instead of the 32-bit registers we have in the regular microarchitecture.
9) The instructions address is only 5-bit width. This means the instruction memory can store a maximum of 32 instructions (as the instruction memory is word addressable).
10) In addition to storing the ALU result in the instruction destination register (rd), you also need to display the result on two 7-Segment displays. You have already designed the ALU in lab4 to handle this task.

Assignment 1
In reference to the microarchitecture shown in Figure 2, explain the meaning of each number or range of numbers written next to the following inputs: **funct3**, **funct7**, **rs1**, **rs2**, and **rd**.

3. Microarchitecture Modules Implementation

As shown in Figure 2, the microarchitecture consists of the following four modules:

- The program counter (PC)
- The control unit.
- The instruction memory
- The register file
- The ALU

The control unit is the combinational circuits from the pre-lab assignment. For the ALU module, you will use the one you implemented in Lab 4. As for the PC, you need to configure it as an up only plain binary LPM_COUNTER with 5 bits output and an "asynchronous" clear input. The implementation and interface of both the instruction memory and register file are available for download from the lab page on Canvas and they are explained in the following sub-sections.

3.1 The Instruction Memory

The following files contain the block diagram and implementation of the instruction memory module and are available on the lab page on Canvas:

- `InstructionMem.bsf`
- `InstructionMem.v`

The `.v` file is the implementation written in the hardware description language (HDL) Verilog. The code is shown in Figure 3, which is explained below.

```
1   module InstructionMem (
2           output [31 : 0]  Instr,
3           input  [4 : 0]   A  );
4
5    //32-bit x 32 words memory
6    reg    [31:0] Instructions[31:0];
7
8    initial
9    begin
10    //Start loading instructions from the mycode.hex file to the
11    // instructions memory starting from address 0
12    $readmemh("C:/temp/mycode.hex", Instructions, 0, 30);
13                      //limit program size to 30 instructions
14
15  end
16
17  //Asynchronous reading
18  assign Instr = Instructions[A];
19
20 endmodule
```

Figure 3: Instruction memory implementation using Verilog

Lines 1-3 show that the instruction memory module receives as input a 5-bit address **A**. It returns asynchronously in its **`Instr`** output the 32-bit content of that address, as shown in line 18. Line 6 defines the memory contents as 32 entries of 32 bits. Every time the module is created in the board, it is initialized (lines 8-15) with the content of the file `mycode.hex` (make sure the file path matches the one shown in line 12).

Before uploading your project to the board, you need to save in the `mycode.hex` file the hexadecimal codes of the instructions to be executed by our compact processor. First write your code as RISC assembly in RARs. After assembling your code using and to save it in Hexadecimal format, click on `File` → Dump `Memory` → make sure the address range is the range of the code then select from the drop-down menu `Hexadecimal Text` → Click on Dump To `File` → save to the `mycode.hex` file.

Note: as shown in line 12, your program will be loaded in the instruction memory starting from address 0 and as the instruction memory asynchronously reads out the instruction at the input address, then once you start your processor, it will automatically execute the program first instruction. This will also happen when you clear register PC. Therefore, it is recommended to have as your program first instruction an instruction that does not change the content of any registers such as `add x0,x0,x0`

3.2 The Register File

The following files contain the block diagram and implementation of the register file module and are available on the lab page on Canvas:

- `RegisterFile.bsf`
- `RegisterFile.v`

The `.v` file is the implementation written in the hardware description language (HDL) Verilog. The code is shown in Figure 4, which is explained below.

```
1  module RegisterFile (
2            output [3 : 0]  SrcA, SrcB,
3            input  [3 : 0]  wd,
4            input  [4 : 0]  rs1, rs2, rd,
5            input  clk );
6
7    //4-bit x 32 words memory
8    reg [3:0] Registers[31:0];
9
10  //Asynchronous reading
11  assign SrcA = Registers[rs1];
12  assign SrcB = Registers[rs2];
13
14  //Initialize all registers
15  integer i = 0;
16  initial
17  begin
18    Registers[0] = 0; //Register x0 in RISC always has the value zero
19    for (i = 1; i < 32; i=i+1) //Do not change content of register x0
20        Registers[i] = i & 2'b11;  //store in each register a value from 0 to 3
21  end
22
23  //Synchronous writing and no writing to x0
24  always @ (posedge clk) begin
25    if ((rd != 5'b00000)) Registers[rd] = wd;
26  end
27 endmodule
```

Figure 4: Register File implementation using Verilog

Lines 1-5 show that the register file module receives five inputs. Three of them are the three 5-bit addresses, `rs1`, `rs2`, and `rd`. Another input is the 4-bit data input, `wd`, to be written to the destination address `rd`. The last input is the 1-bit `clk` input. The positive edge of `clk` triggers writing `wd` to the destination register, as shown in lines 24-26. The register file has two outputs `SrcA` and `SrcB`, . These outputs reflect the contents of the registers indicated by `rs1` and `rs2` respectively. Reading from the register file is done asynchronously (without the `clk` control) as shown in lines 11-12.

Line 8 defines the register file contents as 32 registers; each has 4 bits. Every time the module is created in the board, it is initialized (lines 15-21) with the values between 0 and 3 (depending on the register number). As shown in line 18 and line 25, RISC-V register number zero must always contain value 0.

Assignment 2 Assemble the following RISC-V program in RARS and include its corresponding hexadecimal code in your report.

```
.text
 main:
    add x0, x0, x0
    add x11, x3, x3
    sub x12, x1, x11
    and x13, x1, x12
    or  x14, x0, x12
```

4. The Single Cycle Processor Project

Now you are ready to implement your compact RISC-V single-cycle processor as follows:

a) Create a new folder for a new project named **SingleCycleProcessor** and, from the previous **Four_bit_ALU_7seg** project folder copy the following files to the new project folder:
   - `Bin2BCD.bsf`, `Bin2BCD.bdf`
   - `display7_en.bsf`, `display7_en.bdf`
   - `four_bit_add_sub.bsf`, `four_bit_add_sub.bdf`
   - `Four_bit_ALU.bsf`, `Four_bit_ALU.bdf`
   - `Four_bit_ALU_7seg.bsf`, `Four_bit_ALU_7seg.bdf`
   - `full_adder.bsf`, `full_adder.bdf`
   - `half_adder.bsf`, `half_adder.bdf`
   - `Mux4bit2to1.bsf`, `Mux4bit2to1.v`
   - `Mux4bit4to1.bsf`, `Mux4bit4to1.v`

b) From Lab 7 page on Canvas, copy the following files to the new project folder:
   - `InstructionMem.bsf`,
   - `InstructionMem.v`
   - `RegisterFile.bsf`
   - `RegisterFile.v`

c) Open the `InstructionMem.v` file on Quartus, then go to the **Project** menu → click **Add Current File to Project**.

d) Open the **RegisterFile.v** file on Quartus, then go to the **Project** menu → click **Add Current File to Project**.

e) On *Quartus Schematic* add one instance from each of the following modules and connect them as shown in Figure 2:
   LPM_COUNTER, **Instruction Memory**, **Register File**, **Four_bit_ALU_7seg**, and your combinational circuits from the pre-lab assignment, which is used to control the ALU.

Assignment 3

Use RARS to write a RISC-V assembly program of at least 10 instructions following the registers numbering as in the example given Assignment 2. Your program must include a variety of the four instructions that our compact processor executes, and it must process the contents of at least 15 different registers, including register x0.

Include the program in your report and trace it manually showing, for each instruction, the result to be stored in the destination register. Use RARS to save the Hexadecimal code of your program in file **`mycode.hex`**. Upload your design to the DE1-SOC board and verify that the result of executing each instruction matches your manual tracing.

Record a 1-minute video of the execution of the program on the board and upload it to the lab assignment on Canvas. Take a screen capture of your design and include it in your report.

5. Lab Submission Instructions

- The lab is intended to be implemented by each **individual** student. It is also allowed for a **maximum of two students** to work in the lab together. If you decide to work in a group of two students, **both students** must **check off** the working lab assignments with a lab instructor and only **one student** must complete the lab submission as explained in the lab assignment on Canvas.
- Write your lab report following the **report template** provided on Canvas. You are required to include the circuit schematics for all assignments and if the assignment requires logic to be derived, then the truth table, K-map, and Boolean expression for the outputs must also be included in the report.
- If you are working with a partner, write both names/IDs on the cover page. It is **prohibited** to include a partner's name on a lab report cover page if that partner does not actively participate in implementing the lab.
- The pre-lab questions prepare you for the challenges you will be presented with in the actual lab. Have their answers ready before the lab session and include these answers in the lab report together with the lab assignments.
- For each lab, submit the following on Canvas before the announced due date/time (only submissions on Canvas are accepted):
    - The lab report as a single Word or PDF document.
    - Any files the assignments might ask for (e.g., a demonstration video file)
    - Submit each of the above files separately (do not upload compressed files).
- You can submit multiple attempts for this lab; however, only what you submit in the last attempt will be graded (i.e., all required reports and files must be included in this last attempt).
- **Late submission** of any lab assignment after the given due date and time will be penalized at 1 point per every late hour. No assignment submission will be accepted 48 hours after its due date/time.
- **Successful submission**: it is your responsibility to verify that you submitted successfully all the required files for the lab. First, to verify that your files were submitted, go to the assignment details page, and make sure that it says "Successfully submitted" under grade. Second, you need to validate your submission details by clicking the Submission and Rubric link on that page.
- **Deliberate plagiarism** in any assignment is an extremely serious offense that may result in failing the course. Plagiarism includes submitting answers that are not yours (e.g., copied from someone else or from the Internet). Copying materials from the course slides posted on Canvas is not considered plagiarism. **Warning**: Copying older Quartus projects often leads to unexpected behavior.